

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
УЛЬЯНОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

Факультет информационных систем и технологий
Кафедра «Измерительно-вычислительные комплексы»

**Отчет по лабораторной работе №6
по дисциплине «Системы искусственного интеллекта»**

Выполнил:
студент группы ИСТбд-31 Коняхин Н.С

Проверил:
Преподаватель Шишкин В.В.

Ульяновск, 2026

Цель:

Создать игру PACMAN с AI агентом в роли колобка и двух приведений под управлением человека. Произвести обучение агента методом «Обучение с подкреплением».

1. Выбор алгоритма

DQN (Deep Q-Network). Выбран как базовый, наиболее изученный и наглядный алгоритм RL для учебных задач. Он объединяет табличный Q-learning с нейросетевой аппроксимацией, демонстрируя ключевые механизмы: experience replay, target network, ϵ -greedy стратегию и минимизацию ошибки Беллмана. Идеально ложится на дискретное пространство действий (4 направления) и не требует сложных архитектур (CNN/Policy Gradient), что упрощает отладку и понимание фундаментальных принципов RL.

2. тип пространства

Действий: spaces.Discrete(4) (0:вверх, 1:вниз, 2:влево, 3:вправо). Соответствует механике перемещения в клеточном лабиринте, минимизирует размер выхода сети и упрощает выбор действия через `argmax`.

Наблюдений: spaces.Box(low=0.0, high=1.0, shape=(11,), dtype=np.float32). Вместо raw-пикселей используется нормализованный вектор фиксированной длины. Это резко снижает вычислительные затраты, ускоряет обучение на CPU и делает задачу сходимой для простой полносвязной сети, сохраняя при этом всю необходимую для навигации информацию.

3. Какие данные получает агент

Агент получает вектор из 11 float-значений в диапазоне [0.0, 1.0]:

6 значений: нормализованные координаты Пакмана и двух призраков ($x/(width-1)$, $y/(height-1)$).

1 значение: бинарный индикатор режима страха (1.0 если активен таймер после поедания энерджайзера, иначе 0.0).

4 значения: флаги проходимости соседних клеток (1 если в направлении нет стены, 0 если стена). Это компактное представление даёт агенту точную позицию, контекст опасности и доступные варианты хода без избыточных данных.

4. параметры для DQN

HIDDEN_SIZE = 128: Достаточно для аппроксимации Q-функции при 11-мерном входе, предотвращает переобучение и обеспечивает быструю работу на CPU.

- LEARNING_RATE = 0.001: Стандартное значение для оптимизатора Adam; даёт стабильную сходимость без резких осцилляций весов.
- GAMMA = 0.99: Высокий коэффициент дисконтирования мотивирует агента учитывать долгосрочную выгоду (критично для стратегии сбора всех точек).

- EPS_START=1.0 → EPS_END=0.1, EPS_DECAY=200: Экспоненциальное затухание гарантирует активное исследование в начале и плавный переход к эксплуатации к ~200-му эпизоду.
- BATCH_SIZE=64, MEMORY_SIZE=10000: Типичные учебные значения. Буфер в 10k переходов покрывает вариативность малого лабиринта, батч 64 обеспечивает стабильные градиенты без перегрузки памяти.
- TARGET_UPDATE = 10: Обновление target-сети каждые 10 эпизодов балансирует стабильность целевых значений и адаптивность обучения.
- N_EPISODES=1000, MAX_STEPS_EPISODE=500: Достаточно для сходимости на карте 12x12; ограничение шагов предотвращает заикливание и задаёт чёткий горизонт эпизода.

5. Выбор библиотек

Проект использует Pygame для отрисовки графики и меню, Gymnasium для стандартизации RL-окружения и PyTorch для построения и обучения нейросети DQN. Дополнительно NumPy обрабатывает данные состояний, а collections.deque хранит буфер опыта.

6. Описание Игры

Правила игры идентичны оригинальному PACMAN человек управляет приведениями а не коломком. Пользователь может запустить обучение нейросети с рандомными ботами. Или начать игру против натренированного ИИ-Пакмана.

Вывод:

Проект успешно демонстрирует применение алгоритма DQN для обучения агента в игровой среде. Интеграция Gymnasium, PyTorch и Pygame создаёт удобную образовательную платформу для изучения основ обучения с подкреплением. Реализованная архитектура обеспечивает стабильную сходимость модели и поддерживает интерактивный режим игры против ИИ.

7.Код

```

import pygame
import numpy as np
import random
import gymnasium as gym
from gymnasium import spaces
import torch
import torch.nn as nn
import torch.optim as optim
from collections import deque
import os

# ----- Константы -----
CELL_SIZE = 30
MAZE_WIDTH = 12 # количество столбцов
MAZE_HEIGHT = 12 # количество строк
WINDOW_WIDTH = MAZE_WIDTH * CELL_SIZE
WINDOW_HEIGHT = MAZE_HEIGHT * CELL_SIZE
FPS = 30

# Цвета
BLACK = (0, 0, 0)
WHITE = (255, 255, 255)
BLUE = (0, 0, 255)
YELLOW = (255, 255, 0)
RED = (255, 0, 0)
PINK = (255, 105, 180)
GREEN = (0, 255, 0)
WALL_COLOR = (0, 0, 139)
PELLET_COLOR = (255, 255, 255)
POWER_PELLET_COLOR = (255, 255, 0)
BUTTON_COLOR = (100, 100, 100)
BUTTON_HOVER_COLOR = (150, 150, 150)

# Лабиринт: 0 - пусто, 1 - стена, 2 - точка, 3 - усиливающая точка
MAZE = [
    [1,1,1,1,1,1,1,1,1,1,1,1],
    [1,2,2,2,2,2,1,2,2,2,2,1],
    [1,2,1,1,1,2,1,2,1,1,2,1],
    [1,3,1,0,1,2,2,2,2,2,2,1],
    [1,2,2,2,2,1,1,0,1,1,2,1],
    [1,2,1,1,2,0,0,0,0,2,2,1],
    [1,2,2,2,2,1,2,0,2,1,2,1],
    [1,2,1,1,2,1,2,1,2,1,2,1],
    [1,2,2,2,2,2,2,2,2,2,3,1],
    [1,2,1,1,2,1,2,1,1,1,2,1],
    [1,2,2,2,2,2,2,2,2,2,2,1],
    [1,1,1,1,1,1,1,1,1,1,1,1]
]

# Стартовые позиции
PACMAN_START = (5, 6) # строка, столбец

```

```
GHOST1_START = (5, 5)
```

```
GHOST2_START = (6, 5)
```

```
SCARED_DURATION = 20 # шагов уязвимости после поедания энерджайзера
```

```
# Награды
```

```
REWARD_PELLET = 10
```

```
REWARD_POWER = 50
```

```
REWARD_EAT_GHOST = 200
```

```
REWARD_DEATH = -100
```

```
REWARD_STEP = -1
```

```
REWARD_WIN = 500
```

```
# DQN параметры
```

```
STATE_SIZE = 11 # pac_x, pac_y, g1_x, g1_y, g2_x, g2_y, scared, wall_up, wall_down,  
wall_left, wall_right
```

```
ACTION_SIZE = 4
```

```
HIDDEN_SIZE = 128
```

```
LEARNING_RATE = 0.001
```

```
GAMMA = 0.99
```

```
EPS_START = 1.0
```

```
EPS_END = 0.1
```

```
EPS_DECAY = 200
```

```
BATCH_SIZE = 64
```

```
MEMORY_SIZE = 10000
```

```
TARGET_UPDATE = 10
```

```
N_EPISODES = 1000
```

```
MAX_STEPS_EPISODE = 500
```

```
# ----- Окружение Gymnasium -----
```

```
class PacmanEnv(gym.Env):
```

```
    metadata = {'render.modes': ['human']}
```

```
    def __init__(self, render_mode=None):
```

```
        super().__init__()
```

```
        self.render_mode = render_mode
```

```
        self.maze = [row[:] for row in MAZE] # статическая стена
```

```
        self.pellets = None # копия, будет инициализирована в reset
```

```
        self.pacman_pos = PACMAN_START
```

```
        self.ghost1_pos = GHOST1_START
```

```
        self.ghost2_pos = GHOST2_START
```

```
        self.scared_timer = 0
```

```
    # Пространства действий и наблюдений
```

```
    self.action_space = spaces.Discrete(4)
```

```
    # наблюдение: координаты (0-1) и индикаторы
```

```
    self.observation_space = spaces.Box(low=0.0, high=1.0,
```

```
        shape=(STATE_SIZE,),
```

```
        dtype=np.float32)
```

```
    # для внутреннего использования
```

```
    self.action_to_dir = {0: (-1, 0), 1: (1, 0), 2: (0, -1), 3: (0, 1)}
```

```
def reset(self, seed=None, options=None):
```

```
    super().reset(seed=seed)
```

```
    # восстанавливаем точки
```

```
    self.pellets = [row[:] for row in MAZE]
```

```
    self.pacman_pos = PACMAN_START
```

```
    self.ghost1_pos = GHOST1_START
```

```
    self.ghost2_pos = GHOST2_START
```

```
    self.scared_timer = 0
```

```
    return self._get_obs(), {}
```

```
def step(self, action_pacman, action_ghosts=None):
```

```
    # action_ghosts - список/кортеж из двух действий для призраков (0-3), или None для случайных
```

```
    reward = REWARD_STEP
```

```
    terminated = False
```

```
    truncated = False
```

```
    # 1. Движение Пакмана
```

```
    dr, dc = self.action_to_dir[action_pacman]
```

```
    new_r = self.pacman_pos[0] + dr
```

```
    new_c = self.pacman_pos[1] + dc
```

```
    if self._is_walkable(new_r, new_c):
```

```
        self.pacman_pos = (new_r, new_c)
```

```
    # 2. Поедание точек
```

```
    r, c = self.pacman_pos
```

```
    if self.pellets[r][c] == 2:
```

```
        self.pellets[r][c] = 0
```

```
        reward += REWARD_PELLET
```

```
    elif self.pellets[r][c] == 3:
```

```
        self.pellets[r][c] = 0
```

```
        reward += REWARD_POWER
```

```
        self.scared_timer = SCARED_DURATION
```

```
    # 3. Движение призраков
```

```
    if action_ghosts is None:
```

```
        # случайные действия
```

```
        action_ghosts = [self._random_ghost_action(self.ghost1_pos),
```

```
                        self._random_ghost_action(self.ghost2_pos)]
```

```
    self._move_ghost(1, action_ghosts[0])
```

```
    self._move_ghost(2, action_ghosts[1])
```

```
    # 4. Проверка столкновений
```

```
    if self.scared_timer > 0:
```

```
        # Пакман может съесть призрака
```

```
        if self.pacman_pos == self.ghost1_pos:
```

```
            reward += REWARD_EAT_GHOST
```

```
            self.ghost1_pos = GHOST1_START # возрождение
```

```
        if self.pacman_pos == self.ghost2_pos:
```

```
            reward += REWARD_EAT_GHOST
```

```
            self.ghost2_pos = GHOST2_START
```

```

else:
    if self.pacman_pos == self.ghost1_pos or self.pacman_pos == self.ghost2_pos:
        reward = REWARD_DEATH
        terminated = True

```

```

# уменьшаем таймер страха

```

```

if self.scared_timer > 0:
    self.scared_timer -= 1

```

```

# 5. Проверка победы

```

```

pellets_left = sum(cell in (2,3) for row in self.pellets for cell in row)
if pellets_left == 0:
    reward += REWARD_WIN
    terminated = True

```

```

# 6. Ограничение шагов (truncated)

```

```

# отслеживание шагов делается снаружи, здесь не используется

```

```

obs = self._get_obs()
info = {}
return obs, reward, terminated, truncated, info

```

```

def _get_obs(self):

```

```

    pr, pc = self.pacman_pos
    g1r, g1c = self.ghost1_pos
    g2r, g2c = self.ghost2_pos
    scared = 1.0 if self.scared_timer > 0 else 0.0

```

```

# нормализуем координаты

```

```

    norm_pr = pr / (MAZE_HEIGHT - 1)
    norm_pc = pc / (MAZE_WIDTH - 1)
    norm_g1r = g1r / (MAZE_HEIGHT - 1)
    norm_g1c = g1c / (MAZE_WIDTH - 1)
    norm_g2r = g2r / (MAZE_HEIGHT - 1)
    norm_g2c = g2c / (MAZE_WIDTH - 1)

```

```

# доступность направлений (0 - стена, 1 - свободно)

```

```

    wall_up = 0 if not self._is_walkable(pr-1, pc) else 1
    wall_down = 0 if not self._is_walkable(pr+1, pc) else 1
    wall_left = 0 if not self._is_walkable(pr, pc-1) else 1
    wall_right = 0 if not self._is_walkable(pr, pc+1) else 1

```

```

    return np.array([norm_pr, norm_pc,
                     norm_g1r, norm_g1c,
                     norm_g2r, norm_g2c,
                     scared,
                     wall_up, wall_down, wall_left, wall_right],
                    dtype=np.float32)

```

```

def _is_walkable(self, r, c):

```

```

    if r < 0 or r >= MAZE_HEIGHT or c < 0 or c >= MAZE_WIDTH:
        return False

```

```

return MAZE[r][c] != 1 # не стена (в оригинальном лабиринте стены только 1)

def _random_ghost_action(self, pos):
    r, c = pos
    valid = []
    for act in range(4):
        dr, dc = self.action_to_dir[act]
        if self._is_walkable(r+dr, c+dc):
            valid.append(act)
    if not valid:
        return 0 # на месте (не должно случаться)
    return random.choice(valid)

def _move_ghost(self, ghost_id, action):
    if action == -1: # призрак не двигается
        return
    if ghost_id == 1:
        pos = self.ghost1_pos
    else:
        pos = self.ghost2_pos
    dr, dc = self.action_to_dir[action]
    new_r = pos[0] + dr
    new_c = pos[1] + dc
    if self._is_walkable(new_r, new_c):
        if ghost_id == 1:
            self.ghost1_pos = (new_r, new_c)
        else:
            self.ghost2_pos = (new_r, new_c)

def render(self):
    if self.render_mode == 'human':
        screen = pygame.display.get_surface()
        if screen is None:
            return
        screen.fill(BLACK)
        # рисуем лабиринт
        for r in range(MAZE_HEIGHT):
            for c in range(MAZE_WIDTH):
                x = c * CELL_SIZE
                y = r * CELL_SIZE
                if MAZE[r][c] == 1: # стена
                    pygame.draw.rect(screen, WALL_COLOR, (x, y, CELL_SIZE, CELL_SIZE))
                elif self.pellets[r][c] == 2: # точка
                    pygame.draw.circle(screen, PELLET_COLOR,
                                      (x + CELL_SIZE//2, y + CELL_SIZE//2), 4)
                elif self.pellets[r][c] == 3: # энерджайзер
                    pygame.draw.circle(screen, POWER_PELLET_COLOR,
                                      (x + CELL_SIZE//2, y + CELL_SIZE//2), 8)
        # Пакман
        pr, pc = self.pacman_pos
        pygame.draw.circle(screen, YELLOW,
                          (pc*CELL_SIZE + CELL_SIZE//2, pr*CELL_SIZE + CELL_SIZE//2),

```



```

        CELL_SIZE//2 - 2)
# Призраки
g1r, g1c = self.ghost1_pos
g2r, g2c = self.ghost2_pos
ghost_color = BLUE if self.scared_timer > 0 else RED
pygame.draw.circle(screen, ghost_color,
                    (g1c*CELL_SIZE + CELL_SIZE//2, g1r*CELL_SIZE + CELL_SIZE//2),
                    CELL_SIZE//2 - 2)
pygame.draw.circle(screen, ghost_color,
                    (g2c*CELL_SIZE + CELL_SIZE//2, g2r*CELL_SIZE + CELL_SIZE//2),
                    CELL_SIZE//2 - 2)
pygame.display.flip()

# ----- DQN Агент -----
class DQN(nn.Module):
    def __init__(self, input_dim, output_dim, hidden_dim):
        super().__init__()
        self.fc = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, output_dim)
        )

    def forward(self, x):
        return self.fc(x)

class DQNAgent:
    def __init__(self):
        self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        self.policy_net = DQN(STATE_SIZE, ACTION_SIZE,
HIDDEN_SIZE).to(self.device)
        self.target_net = DQN(STATE_SIZE, ACTION_SIZE, HIDDEN_SIZE).to(self.device)
        self.target_net.load_state_dict(self.policy_net.state_dict())
        self.target_net.eval()
        self.optimizer = optim.Adam(self.policy_net.parameters(), lr=LEARNING_RATE)
        self.memory = deque(maxlen=MEMORY_SIZE)
        self.epsilon = EPS_START
        self.steps_done = 0

    def act(self, state, evaluate=False):
        if not evaluate and random.random() < self.epsilon:
            return random.randrange(ACTION_SIZE)
        state_tensor = torch.FloatTensor(state).unsqueeze(0).to(self.device)
        with torch.no_grad():
            q_values = self.policy_net(state_tensor)
        return q_values.argmax().item()

    def remember(self, state, action, reward, next_state, done):
        self.memory.append((state, action, reward, next_state, done))

```

```

def replay(self):
    if len(self.memory) < BATCH_SIZE:
        return
    batch = random.sample(self.memory, BATCH_SIZE)
    states, actions, rewards, next_states, dones = zip(*batch)

    states = torch.FloatTensor(states).to(self.device)
    actions = torch.LongTensor(actions).unsqueeze(1).to(self.device)
    rewards = torch.FloatTensor(rewards).unsqueeze(1).to(self.device)
    next_states = torch.FloatTensor(next_states).to(self.device)
    dones = torch.FloatTensor(dones).unsqueeze(1).to(self.device)

    q_values = self.policy_net(states).gather(1, actions)
    with torch.no_grad():
        next_actions = self.policy_net(next_states).argmax(1).unsqueeze(1)
        next_q_values = self.target_net(next_states).gather(1, next_actions)
        target = rewards + GAMMA * next_q_values * (1 - dones)
    loss = nn.MSELoss()(q_values, target)
    self.optimizer.zero_grad()
    loss.backward()
    self.optimizer.step()

def update_target(self):
    self.target_net.load_state_dict(self.policy_net.state_dict())

def decay_epsilon(self, episode):
    self.epsilon = EPS_END + (EPS_START - EPS_END) * np.exp(-episode / EPS_DECAY)

def save(self, path='pacman_dqn.pth'):
    torch.save(self.policy_net.state_dict(), path)

def load(self, path='pacman_dqn.pth'):
    self.policy_net.load_state_dict(torch.load(path, map_location=self.device))
    self.target_net.load_state_dict(self.policy_net.state_dict())
    self.policy_net.eval()
    self.target_net.eval()

# ----- Режим обучения -----
def train_agent():
    pygame.init()
    screen = pygame.display.set_mode((WINDOW_WIDTH, WINDOW_HEIGHT))
    pygame.display.set_caption("Пас-Ман: Обучение AI")
    clock = pygame.time.Clock()

    env = PacmanEnv(render_mode='human')
    agent = DQNAgent()
    episode_rewards = []

    for episode in range(1, N_EPISODES+1):
        state, _ = env.reset()
        total_reward = 0
        steps = 0

```

```

done = False

while not done and steps < MAX_STEPS_EPISODE:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            return

    action = agent.act(state)
    next_state, reward, terminated, truncated, _ = env.step(action)
    agent.remember(state, action, reward, next_state, terminated or truncated)
    agent.replay()
    state = next_state
    total_reward += reward
    steps += 1
    done = terminated or truncated

    env.render()
    clock.tick(FPS)

    agent.decay_epsilon(episode)
    if episode % TARGET_UPDATE == 0:
        agent.update_target()
    episode_rewards.append(total_reward)
    if episode % 50 == 0:
        avg_reward = np.mean(episode_rewards[-50:])
        print(f"Эпизод {episode:4d}, Средняя награда за 50: {avg_reward:.2f}, Epsilon:
{agent.epsilon:.3f}")

    agent.save('pacman_dqn.pth')
    print("Обучение завершено, модель сохранена как pacman_dqn.pth")
    pygame.quit()

# ----- Режим игры двух человек против AI -----
def play_human_vs_ai():
    if not os.path.exists('pacman_dqn.pth'):
        print("Модель не найдена! Сначала обучите агента.")
        return

    pygame.init()
    screen = pygame.display.set_mode((WINDOW_WIDTH, WINDOW_HEIGHT))
    pygame.display.set_caption("Пас-Ман: Люди против AI")
    clock = pygame.time.Clock()

    env = PacmanEnv(render_mode='human')
    agent = DQNAgent()
    agent.load('pacman_dqn.pth')
    agent.epsilon = 0 # без исследования

    state, _ = env.reset()
    done = False
    game_result = None

```

```

while not done:
    # обработка ввода
    ghost1_action = None
    ghost2_action = None
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            return
    # клавиши для перезапуска после окончания игры обрабатываем отдельно

    keys = pygame.key.get_pressed()
    # Призрак 1: стрелки
    if keys[pygame.K_UP]:
        ghost1_action = 0
    elif keys[pygame.K_DOWN]:
        ghost1_action = 1
    elif keys[pygame.K_LEFT]:
        ghost1_action = 2
    elif keys[pygame.K_RIGHT]:
        ghost1_action = 3

    # Призрак 2: WASD
    if keys[pygame.K_w]:
        ghost2_action = 0
    elif keys[pygame.K_s]:
        ghost2_action = 1
    elif keys[pygame.K_a]:
        ghost2_action = 2
    elif keys[pygame.K_d]:
        ghost2_action = 3

    # Действия AI для Пакмана
    pacman_action = agent.act(state, evaluate=True)

    # Если игрок не нажал направление, призрак не двигается (или случайно?)
    # Пусть призрак стоит на месте, если действие не задано (action=None
    # обрабатывается как случайное в step,
    # но в режиме игры нужно, чтобы без нажатия призрак не двигался. Изменим
    # логику: если None, то оставаться на месте.
    # Но в step по умолчанию action_ghosts=None включает случайное движение.
    # Передадим список действий, где None означает без движения.
    actions = [ghost1_action if ghost1_action is not None else -1, # -1 будем считать
    "стоять"
                ghost2_action if ghost2_action is not None else -1]

    # Модифицируем вызов step, чтобы передать список, но с поддержкой -1 как
    "стоять на месте".
    # Поэтому напишем кастомный шаг с ручным движением призраков, или добавим
    параметр в step.
    # Легче вызвать env.step с переданными действиями, но для "стоять" нужно
    проверять.

```

```

# Изменим _move_ghost, чтобы он не двигался, если action == -1.
# Для этого в методе step будем проверять: если action_ghosts не None, то для
каждого действия, если -1, не двигаем.
# Я добавлю в step обработку -1.

next_state, reward, terminated, truncated, _ = env.step(pacman_action,
action_ghosts=actions)
state = next_state
done = terminated or truncated

env.render()
clock.tick(FPS)

if done:
    # Сохраняем результат для отображения
    if terminated and reward >= REWARD_WIN:
        game_result = "Победа! Все точки собраны."
    else:
        game_result = "Поражение! Пакман пойман."
    # ждём клавишу
    waiting = True
    while waiting:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                pygame.quit()
                return
            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_r:
                    # перезапуск игры
                    state, _ = env.reset()
                    done = False
                    game_result = None
                    waiting = False
                elif event.key == pygame.K_m:
                    # возврат в меню
                    waiting = False
                    pygame.quit()
                    return
        # можно отобразить сообщение на экране
        screen.fill(BLACK)
        font = pygame.font.Font(None, 36)
        text = font.render(game_result, True, WHITE)
        text_rect = text.get_rect(center=(WINDOW_WIDTH//2, WINDOW_HEIGHT//2 -
40))
        screen.blit(text, text_rect)
        text2 = font.render("R - ещё раз, M - меню", True, WHITE)
        text2_rect = text2.get_rect(center=(WINDOW_WIDTH//2, WINDOW_HEIGHT//2
+ 10))
        screen.blit(text2, text2_rect)
        pygame.display.flip()
        clock.tick(15)
    if not done and game_result is None:

```

```
        continue
    else:
        break # выход после M
```

```
pygame.quit()
```

```
# Обновим метод step окружения, чтобы он мог обрабатывать action=-1 (стоять на месте)
```

```
def patched_step(self, action_pacman, action_ghosts=None):
    # см. выше, но с поддержкой -1
    # переопределим метод (лучше создать новый класс или модифицировать исходный)
    # Для простоты я изменю класс PacmanEnv, добавив проверку.
    pass
```

```
# Я просто добавлю в оригинальный step поддержку -1, обновив код класса.
```

```
# ----- Главное меню -----
```

```
def main_menu():
    pygame.init()
    screen = pygame.display.set_mode((400, 300))
    pygame.display.set_caption("Pac-Man AI")
    clock = pygame.time.Clock()
    font = pygame.font.Font(None, 36)

    button_train = pygame.Rect(100, 100, 200, 50)
    button_play = pygame.Rect(100, 180, 200, 50)

    running = True
    while running:
        screen.fill(BLACK)
        mx, my = pygame.mouse.get_pos()
        # кнопка "Обучить"
        if button_train.collidepoint(mx, my):
            pygame.draw.rect(screen, BUTTON_HOVER_COLOR, button_train)
        else:
            pygame.draw.rect(screen, BUTTON_COLOR,
button_train)
        # кнопка "Играть"
        if button_play.collidepoint(mx, my):
            pygame.draw.rect(screen, BUTTON_HOVER_COLOR, button_play)
        else:
            pygame.draw.rect(screen, BUTTON_COLOR, button_play)

        text_train = font.render("Обучить", True, WHITE)
        text_play = font.render("Играть", True, WHITE)
        screen.blit(text_train, (button_train.x+50, button_train.y+10))
        screen.blit(text_play, (button_play.x+55, button_play.y+10))

    pygame.display.flip()

    for event in pygame.event.get():
        if event.type == pygame.QUIT:
```

```

    running = False
elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
    if button_train.collidepoint(event.pos):
        # закрываем меню и запускаем обучение
        pygame.quit()
        train_agent()
        # после обучения возвращаемся в меню
        main_menu()
        return
    elif button_play.collidepoint(event.pos):
        pygame.quit()
        play_human_vs_ai()
        main_menu()
        return
clock.tick(30)

pygame.quit()

# ----- Запуск -----
if __name__ == "__main__":
    main_menu()

```